Reverse Engineering: Key Validation

2026-10-13

Table of contents

1 Introduction **1**

2 Download the File **2**

3 Observe the Program **2**

4 Open in Cutter **2**
- 4.1 Decoding Main - Part 1 . . . 3
- 4.2 Decoding Main - Part 2 . . . 4

5 The Custom Function - `validate(key)` **6**
- 5.1 Read the Validation Function . . . 7
- 5.2 The Return Statement . . . 7
- 5.3 The Loop Contents . . . 9

6 Assignment **11**
- 6.1 The Other Loop . . . 11
- 6.2 The First Character Check . . . 12
- 6.3 The Length Check . . . 12

7 Construct a Valid Key and Verify It **12**

1 Introduction

Software vendors often protect their products with license keys; that is, they make short strings that a program validates before granting access. Understanding how these validators work is a core reverse engineering skill. Rather than patching around the check by reversing a jump (as you did in an earlier lab), your goals in this lab are:

1. To read and understand the validation algorithm well enough to produce keys that genuinely satisfy it.
2. To reconstruct the original C program that produced the `validate` executable.

2 Download the File

The files needed for this lab are located on Canvas as `key-validation-executables.zip`.

Download `key-validation-executables.zip` from Canvas onto your VM and unzip it. Inside you will find one file named `validate-cutter` and several others named `validate-*-*`, where the `*` indicates the architecture and OS that the file was compiled for. You may need to use your virtual machine to run a `validate-*-*` executable.

For reference, we will only use the file titled `validate-cutter` for analysis with Cutter. When asked to run the file, use the appropriate one for your machine's architecture.

For any file you plan on executing, be sure to use `chmod +x [filename]` to make it executable.

3 Observe the Program

Run the program and try a few inputs:

```
./validate-[arch]-[os]
```

Replace `[arch]` and `[os]` with the values in the filename that match your machine.

The program prompts for a license key. Any input you try will likely print `Invalid key.` Your goal is to use Cutter to find a key that the program accepts.

> **No Patching**
>
> Do not modify any instructions or data in the binary. The objective is to understand the algorithm and construct a key that satisfies it as written.

4 Open in Cutter

Open the `validate-cutter` binary in Cutter. At the `Load Options` dialog, leave the defaults and click `Ok`.

4.1 Decoding Main - Part 1

Navigate to the `Graph` tab and select `entry0` (or `main`) from the left panel. This represents the main function.

With respect to the contents of `entry0` (or `main`), you should see that it is short. You should see that there is a block of instructions with three variables at the top (shown as comments, which are marked by a `;`). Note that some of these variables may not be necessary when reconstructing the `C` code. Some example variables you may see are:

- `var_5ch` as an `int64_t`
- `var_58h` as an `int64_t`
- `var_10h` as an `int64_t`

One of these three variables will represent the input given by the user.

Elsewhere within this first block in main, you should be able to spot a string for the `Enter license key:` prompt, the `printf` function being called, the `scanf` function being called, and then some custom function labeled `func.100000470`.

! Reconstructing Main - Part 1

As a group, try to reconstruct some `C` that covers everything discussed so far. Here are some guiding questions to help out. Read through all the questions before answering any, or writing any `C`. You may have to search for some answers online.

1. Are there any variables that need to be created to store inputs?
2. Maybe it has been a while since you've seen `C`. How do we use `scanf`? What do the parameters of `scanf` represent?
3. Which registers does `scanf` typically act on? Which variables are associated with those registers?
4. Why does `str.63s` appear as a string near `scanf`?
5. What does `63s` mean with respect to format strings?
6. Are there any arguments associated with the custom function? Hint: Check immediately before the call to the function to see if any variables are mentioned.

A possible solution follows for you to check your work.

Reconstructing Main - Part 1 Solution

Some possible realizations you may have had are:

1. The `scanf` function acts on register `rdi` for holding the pointer to the format string parameter (e.g., "%d") and `rsi` to hold the address of the variable (e.g., &num) that represents the input.
2. The variable with the label `var_58h` represents the variable that `scanf` acts on since `lea` is acting on it with respect to `rsi`.
3. The string `"63s"` is related to the format string, specifically a format string that has a max width of 63 characters. If you consider the null terminating character (`'\0'`), you get 64 total characters for the string that will store our input.
4. The variable that is acted on by `scanf` is the same variable that is passed into the custom function. Cutter places a comment showing it as `char *arg1`.

Here is some possible C code that covers what we've seen so far:

```c
#include <stdio.h>

int validate(char *key); // defined later

int main(){
    char key[64]; // 64 characters for the key

    // prompt for key
    printf("Enter license key: ");
    scanf("%63s", key);  // store input into key

    // call the custom function
    int result = validate(key);

    // tbd
}
```

4.2 Decoding Main - Part 2

After the custom function is called, the top block in `main` ends with a jump if equal (`je`) command which comes right after a `cmp` instruction and right before a **branch**.

The green line (i.e. the "comparison was equal" branch) leads to a block where the string `Invalid key.` is found, followed by a call to `printf`.

The red line (i.e. the "was not equal" branch) leads to a block where the string `License accepted.` is found, followed by a call to `printf`.

Notice the values being compared immediately before the jump, that is the register and the immediate value (constant).

Reconstructing Main - Part 2

As a group, try to reconstruct some more of the `C` that covers the call to the custom function we are referring to as `validate`. Here are some guiding questions to help out. Read through all the questions before answering any, or writing any `C`. You may have to search for some answers online.

1. What register do custom functions typically store return values in?
    - You might consider exploring the final block of the custom function in the **Graph** view to make some inferences about this value (such as its type and values it takes on)
2. Which two values are being compared?
3. Based on the branching, what value do you think the custom function `validate` returns if it was not successful?

A possible solution follows for you to check your work.

Reconstructing Main - Part 2 Solution

Here are some possible solutions/realizations you may have had.

1. The custom `validate` function is storing its return value in the register `eax`.
2. Open the instructions for the custom function in the graph view. Scroll to the bottom/last block of code where the `ret` instruction lives. You can see that `eax` is asked to store `dword [var_ch]` with the instruction `mov eax, dword [var_ch]`.

   The first block (at the top) of the custom function's graph view shows `[var_ch]` as an `int`.

   Further exploring locations where `var_ch` is set to a value via `mov` inside of the custom function shows that it only takes on the values of `0` and `1`.
3. Back in `main/entry0`, the value stored in `eax` is being compared to the value `0` immediately before the jump.

4. The green path is taken if the two values were equal (i.e. a jump is made). To see the actual jump, investigate the **Disassembly** tab. If the jump is made, then a return of 0 from `validate` means the key was invalid since the `Invalid key.` string is output. A return of anything else means the key was valid.

A possible solution follows for you to check your work so far.

```c
#include <stdio.h>

int validate(char *key); // defined later

int main(){
    char key[64];

    printf("Enter license key: ");
    scanf("%63s", key);

    int result = validate(key);

    // print the proper message
    if (result == 0){            // cmp eax, 0; je
        printf("Invalid key.\n");
    } else {
        printf("License accepted.\n");
    }

    // We are not concerned with what comes after the branches
    // caused by the jump

    return 0;
}
```

We are not concerned with the remainder of main.

5 The Custom Function - `validate(key)`

From now on, we will be working in the `validate` function.

In the **Graph** view, double-click on the memory address of the `call` instruction that invokes the custom function. This should open that function in the **Graph** view.

Alternatively, use the `Functions` panel on the left to see all detected functions and navigate to the one that is not `entry0`, `main`, or a library stub (library stubs usually start with `sym.imp`).

5.1 Read the Validation Function

The validation function is the heart of this lab.

Let's work through it systematically in the **Graph** view. There are various branches throughout because the function will complete multiple checks to validate the license key.

As you have already determined, the function returns `0` if the key is invalid, following the branch toward `Invalid key.` being printed. Only if all checks pass does it return `1` (valid). Be sure to navigate to the **Graph** view once you've opened it.

We will work from the bottom up.

5.2 The Return Statement

Let's start by finding the block where the function returns (this should be at the bottom of the **Graph** view). You should see the instruction named `ret` for return.

Note that the register that stores the return value of a custom function will be labeled `eax` (32-bits) or `rax` (64-bits) depending on the number of bits we are returning.

Also, keep in mind that in x86 assembly a `double word` (or `dword`) is a 32-bit value (regardless of type) even though we typically associate the word `double` with a 64-bit floating point value. In addition, a `quadword` would be 64 bits (regardless of type).

Be sure to note how Cutter is identifying the return value with `dword [var_ch]`.

> **!** Questions
>
> Answer the following questions, take notes, and discuss it with your group. You are building up a description of exactly what a valid key must look like. After discussing with your group what may be happening in this section of the code, read the `Post Discussion` block that follows.
>
> 1. Starting with the return block, follow the arrows back to identify the block where the return variable `var_ch` is set to 1. We will refer to this block as the **success** block.
> 2. Can you identify code immediately before the **success** block that makes a comparison? What comparison/jump is causing program execution to land in this **success** block?

3. You've probably identified a block that has 2 arrows coming into it: one block from higher up in the graph view and another from lower in the graph view.

   What common programming language construct do you think having an arrow come from below is indicative of?

   We will refer to this block that has two arrows coming into it as the **double-in** block.

4. Look at both the parents of the **double-in** block.

   - Is there a common variable label assigned by Cutter that appears across the **double-in** block and its parents?
   - What value is being assigned to this variable in the higher parent?
   - How is this variable being operated on in the lower parent?

5. Can you find a path that might start at the **double-in** block and then end up back at the **double-in** block?

6. Can you write a portion of a code block in `C` that represents what is going on here?

Post Discussion

Now that you've discussed what may be happening with your team, consider the following:

- It appears a loop is occurring.
- The upper parent appears to set some variable, let's say `i`, to 2. (example: `mov dword [var_28h], 2`)
- In a situation where success happens, that loop will execute until `i` reaches 8. Then the code moves on to indicate success has happened and the key has been validated.
- You might have noticed that this variable, `i`, is being incremented by 2 in the lower parent (example: `add eax, 2`).

Perhaps the source for the portion of this function that we have investigated looks something like the following:

```c
int validate(char *key){
    // start of function omitted

    // the loop we've identified
    for (int i=2; i < 8; i+=2){
        // stuff
    }

    return 1; // success
}

// main() is omitted for brevity
```

5.3 The Loop Contents

Now that we've identified a loop, let's analyze what is happening inside of it.

! Guiding Questions

Read through the following guiding questions and discuss your thoughts with your group before moving to the `Post Discussion` section below.

1. Find the **double-in** block at the start of the loop we discussed in the previous section.
2. Are there comparisons happening in the looping path? What values are those comparisons being made against?
    - Note that comments in the code are indicated by a semicolon `;` and could be helpful.
    - Other than being an integer, could these values represent something in some other encoding?
3. Does the looping path ever break off and lead toward a failure scenario (that is, `return 0`)?
4. Is the iterating variable (`i` or `var_28h`) being used within the looping path for anything other than as an iterator? That is, are there additional operations being applied to it near the comparisons?
5. What could `rax` be referring to? Note the label Cutter uses (`qword [s]` or similar) and scroll back to the beginning of the `validate` function to see if there is a hint.

6. Try modifying your previously written C to accommodate your discoveries here.

Post Discussion

Here are some considerations:

- You might have noticed that there are two comparisons happening. One comparison is associated with the value 65 (`0x41`) and the other with 90 (`0x5a`). In ASCII, these values represent the capital letters `'A'` and `'Z'`.
- You may have also noticed that after the first comparison, there is a jump to a path with a return of 0 if the value being compared is less than the character `'A'`.
- After the second comparison, it can jump to failure if it is greater than `'Z'`.
- Thus, the value that is being compared must be between `'A'` and `'Z'`.

At this point you might have a loop that looks something like this.

```
int validate(char *key){
    // start of function omitted

    for (int i=2; i < 8; i+=2){

        // something is between A and Z
        if ( something < 'A' || something > 'Z'){
            return 0; // fail
        }

    }


    return 1; // success
}
```

You may have noticed that the value being compared is placed into `eax`. Right before each comparison, there is a value associated with `rax` and `rcx` placed into `eax`.
Immediately before that, `rcx` takes on the value of the iterator, `i`. Investigation into the very first block of this function shows us that the argument passed into the function is given a label that then ends up in `rax` for these two comparison blocks.
Since the argument being passed into the function is a string that represents the key we are trying to validate, you might assume we are iterating over the characters of this key string resulting in the following code.

```c
int validate(char *key){

    // start of function omitted

    // Characters at index 2, 4, and 6 are
    // capital letters from A to Z
    for (int i=2; i < 8; i+=2){
        if ( key[i] < 'A' || key[i] > 'Z'){
            return 0; // fail
        }
    }

    return 1; // success
}

// main omitted
```

6 Assignment

The rest of the reconstruction of this code in C and determining a valid key is left as an exercise. Your goal is to reconstruct the C program that resulted in this machine code. Each of the sections that follow assists you as you work backward toward the top of the `validate` function.

Note that the **Decompiler** tab in Cutter can be helpful. In the bottom right corner of this tab, check out the options you have for the decompiler. The Ghidra decompiler is rather powerful.

6.1 The Other Loop

Next, look for another loop in the instruction set. Here are some guiding questions and comments to help out.

This loop iterates over specific positions in the key and checks each character. Read the loop carefully to answer these questions:

1. What starting index does the loop use?
2. How much does the index increment each iteration?

3. What range of character values does each character need to fall within? (Look for two `cmp` instructions with constants: one for the lower bound and one for the upper bound.)
4. What arithmetic does the loop perform on each character, and what does it accumulate?

After the loop (and immediately before the letter loop), there will be a comparison of the accumulated value against a constant. That constant is the required sum. Note that the constant will appear in hexadecimal. Convert it to decimal.

6.2 The First Character Check

Before any looping, the function loads the first byte of the key and compares it against a constant. Find this comparison.

The constant will appear as a hexadecimal value in the assembly. Convert it to its ASCII character. That character must be the first character of your key.

6.3 The Length Check

Before the first character check, the first thing the function checks is the length of the input. Look for a call to `strlen`, followed by a comparison instruction, followed by a conditional jump.

What length does the key need to be?

7 Construct a Valid Key and Verify It

Design a key that satisfies the program you wrote and test it against the appropriate `validate-*-*` file for your machine. Recall that the file you examined (`validate-cutter`) will most likely not run on your host or VM. If you were curious, Cutter shows the OS and architecture a compiled file targets on the **Dashboard** tab.

You should see `License accepted.` If you see `Invalid key.`, re-examine each check. One of your digits may be off, or a position may be wrong. Work through the checks one at a time against your key to find the mismatch.